

PROJECT REPORT

1. Abstract Summary

The **Smart MCQ Solver Challenge** aims to develop artificial intelligence models capable of predicting the top three most probable answers for multiple-choice questions. Each question contains a prompt and five answer options, and model performance is evaluated using **Mean Average Precision at 3 (MAP@3)**.

This project explored and compared four different approaches: a **Logistic Regression** baseline, a **custom Multi-Layer Perceptron (MLP)**, a **fine-tuned BERT** model, and a **Retrieval-Augmented Generation (RAG)** pipeline. Each model was evaluated using Accuracy, Macro F1-score, Validation MAP@3, and Kaggle leaderboard performance to analyze their effectiveness in solving the MCQ ranking task.

Among the evaluated models, the **Logistic Regression** baseline achieved the best competition performance with a **Kaggle MAP@3 score of 0.751**, outperforming the more complex deep learning and retrieval-based approaches. The project provided practical experience in implementing and comparing traditional machine learning, deep learning, transformer-based, and retrieval-augmented techniques for multiple-choice question answering, while highlighting the importance of selecting models appropriate to the dataset and evaluation objective.

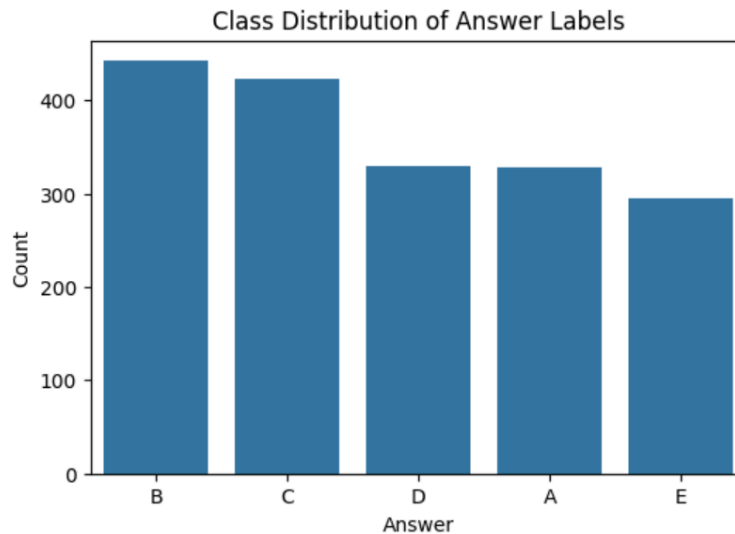
2. Introduction

Multiple Choice Question Answering (MCQA) is an important Natural Language Processing (NLP) task that evaluates a model's ability to understand textual context, reason over information, and identify the most appropriate answer from multiple candidate options. The **Smart MCQ Solver Challenge** provides a benchmark for developing AI models capable of predicting the **top three most probable answers** for each multiple-choice question. The competition dataset consists of a collection of MCQs, where each question contains a prompt and five answer choices labeled **A, B, C, D, and E**. Model performance is evaluated using **Mean Average Precision at 3 (MAP@3)**, which rewards models that rank the correct answer higher in their predictions.

3. Dataset & Preprocessing

The Smart MCQ Solver Challenge dataset consists of multiple-choice questions, where each sample contains a question prompt, five candidate answer options (A–E), and the corresponding correct answer label. The original training dataset contained **2,000** samples with eight attributes (id, prompt, A, B, C, D, E, and answer), while the test dataset contained **500** samples without the answer labels.

Initial exploratory data analysis (EDA) showed that the dataset contained **no missing values**, ensuring that all samples were complete for model training. Duplicate analysis identified **242 duplicate questions**, which were removed to improve data quality and prevent information leakage during model evaluation. After duplicate removal, the final training dataset consisted of **1,817** unique samples.



Although the classes are not perfectly balanced, the distribution remains reasonably uniform across the five answer options, making the dataset suitable for multi-class learning without requiring additional balancing techniques.

Before model training, text preprocessing was applied to all textual fields, including the question prompt and answer options. The preprocessing pipeline converted text to lowercase, removed punctuation and special characters, and normalized whitespace to produce consistent textual representations. The cleaned dataset was then divided into training and validation sets for model development and evaluation.

For the **Logistic Regression** and **MLP** models, the original multi-class dataset was transformed into a binary classification dataset. Each question was expanded into five question–option pairs, where the correct option was assigned a label of **1** and the remaining options were assigned **0**. The question prompt and corresponding answer option were concatenated to form the input text. This transformation enabled binary classifiers to estimate the probability of each candidate answer independently, after which the options belonging to the same question were ranked to generate the final Top-3 predictions required by the competition.

4. Tokenization Strategy

Different text representation techniques were adopted based on the requirements of each model. For the **Logistic Regression** and **MLP** models, text was converted into numerical feature vectors using the **TF-IDF (Term Frequency–Inverse Document Frequency)** vectorizer with unigram and bigram features. This approach efficiently represents word importance and is well suited for traditional machine learning models.

For the **BERT** model, the **WordPiece tokenizer** associated with the pre-trained **bert-base-uncased** model was used through the Hugging Face Transformers library (`BertTokenizer`). WordPiece tokenization splits words into subword units, enabling the model to effectively handle rare or unseen words while preserving contextual meaning. The tokenization pipeline employed **batch processing**, along with **padding** and **truncation**, to produce fixed-length input sequences for efficient model training.

Unlike the previous models, the **RAG** pipeline used dense sentence embeddings generated by the **SentenceTransformer (all-MiniLM-L6-v2)** model for semantic retrieval instead of conventional tokenization. The use of TF-IDF, WordPiece tokenization, and sentence embeddings ensured that each model employed the text representation most suitable for its underlying architecture and learning methodology.

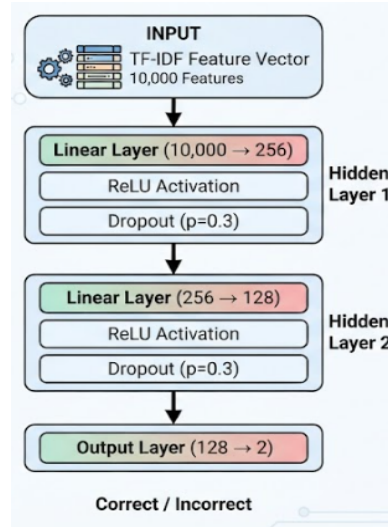
5. Modeling & Experimentation

5.1 Model 1: Logistic Regression (Baseline)

A **Logistic Regression** model was implemented as the baseline using **TF-IDF** features with unigram and bigram representations (`max_features=30000`). Since the competition required ranking the top three answers, the original dataset was transformed into a **binary classification** dataset, where each question–option pair was labeled as correct or incorrect. The model was trained using Scikit-learn's `LogisticRegression` (`max_iter=1000`). During inference, the predicted probability of the positive class was obtained using `predict_proba()`, and the five options belonging to the same question were ranked to generate the final Top-3 predictions.

5.2 Model 2: Custom Deep Learning Model (MLP)

A **Multi-Layer Perceptron (MLP)** was developed from scratch using **PyTorch** to perform binary classification on the expanded dataset. The input layer received **10,000-dimensional TF-IDF feature vectors** generated from the concatenated question prompt and candidate answer option. The network consisted of two fully connected hidden layers with **256** and **128** neurons, followed by **ReLU** activation and **30% dropout** for regularization. The output layer contained **2 neurons**, representing the binary classes (correct/incorrect option). The model was trained using the **Adam** optimizer (`learning rate = 0.001`), **CrossEntropyLoss**, a batch size of **32**, and **10 epochs**.

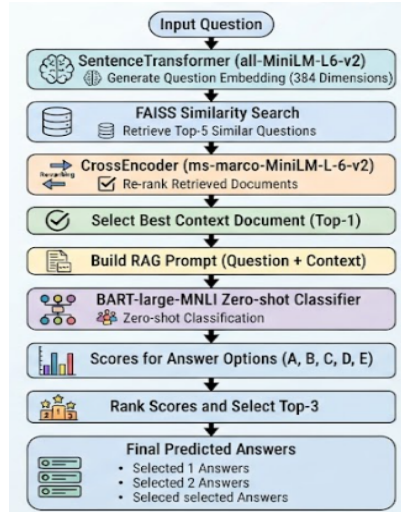


5.3 Model 3: Fine-Tuned Transformer Model (BERT)

A pre-trained **BERT-base-uncased** model was fine-tuned for five-class answer prediction using the Hugging Face Transformers library. BERT employs a **bidirectional Transformer encoder** with a self-attention mechanism to capture contextual relationships between words. Fine-tuning was performed for **4 epochs** using the **AdamW** optimizer with a learning rate of 2×10^{-5} , batch size **16**, weight decay **0.01**, and a warmup ratio of **0.1**. All model parameters were fine-tuned without applying quantization. The final classification layer produced probabilities for the five answer options (A–E), and the option with the highest probability was selected as the predicted answer.

5.4 Model 4: Retrieval-Augmented Generation (RAG)

A **Retrieval-Augmented Generation (RAG)** pipeline was implemented by combining semantic retrieval with zero-shot classification. Question embeddings were generated using **SentenceTransformer (all-MiniLM-L6-v2)** and indexed using **FAISS** to retrieve the **top five** most similar documents. These retrieved documents were then re-ranked using **CrossEncoder (ms-marco-MiniLM-L-6-v2)**, and the highest-ranked document was selected as contextual knowledge. The retrieved context was combined with the input question and passed to **BART-large-MNLI**, which performed zero-shot classification by assigning confidence scores to all five answer options. The options were then ranked based on these scores to generate the final Top-3 predictions.

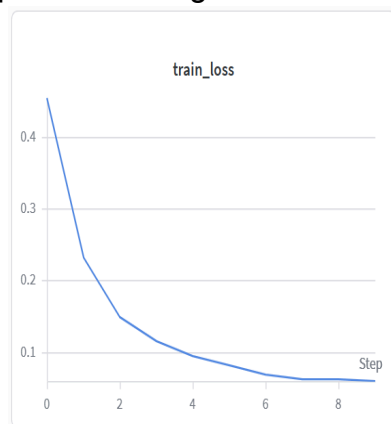


6. Performance & Comparative Analysis

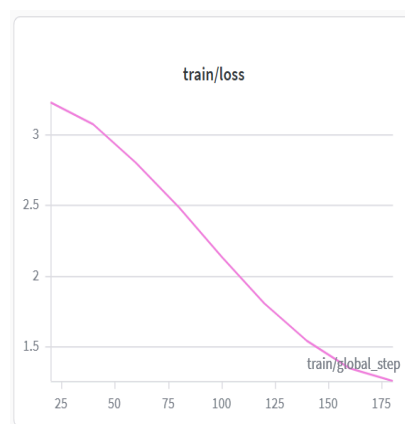
Evaluation Metrics

Model performance was evaluated using **Accuracy**, **Macro F1-score**, and **Mean Average Precision at 3 (MAP@3)**. Accuracy and Macro F1-score were used to measure classification performance on the validation dataset, while **MAP@3** served as the primary evaluation metric for the Kaggle competition by assessing the quality of the ranked Top-3 answer predictions. The final submission using the **Logistic Regression** baseline achieved a **MAP@3 score of 0.751** on the Kaggle public leaderboard, securing an overall **rank of 738**.

The training behavior of the **MLP** and **BERT** models was monitored using **Weights & Biases (W&B)**. Training loss curves demonstrated stable convergence for both models without significant training instability. These visualizations are included to illustrate the optimization process during model training.



MLP



BERT

Model	Accuracy	Macro F1	Validation MAP@3	Kaggle MAP@3
Logistic Regression	0.9810	0.9800	0.9872	0.751
MLP	0.9588	0.9300	0.9725	0.730
BERT	0.9148	0.9106	0.9556	0.732
RAG Pipeline	0.3407	0.3370	0.5472	0.440

7. Conclusion & Future Work

Conclusion

This project explored four different approaches for solving the Smart MCQ Solver Challenge, including Logistic Regression, MLP, BERT, and a Retrieval-Augmented Generation (RAG) pipeline. The experimental results demonstrated that the Logistic Regression baseline achieved the best overall performance with a Kaggle MAP@3 score of 0.751, outperforming the more complex deep learning models. The project provided valuable hands-on experience in text preprocessing, feature engineering, transformer fine-tuning, retrieval-based architectures, and comparative evaluation of machine learning models for multiple-choice question answering.

Challenges Faced

The major challenges included selecting appropriate text representations for different model architectures, preventing overfitting on a relatively small dataset, and improving the generalization performance of transformer and retrieval-based models on the unseen Kaggle test set.

Future Work

Future improvements could include exploring larger pre-trained language models, implementing ensemble techniques, optimizing hyperparameters, and improving retrieval quality within the RAG pipeline. These enhancements may further improve ranking performance on the MCQ answering task.

8. References

1. Hugging Face Transformers Documentation. <https://huggingface.co/docs/transformers>
2. Scikit-learn Documentation. <https://scikit-learn.org/>
3. PyTorch Documentation. <https://pytorch.org/docs/>
4. FAISS Library Documentation. <https://github.com/facebookresearch/faiss>